

LearningSteps Lockdown API Security Runbook

Goal

Protect the LearningSteps FastAPI app with Microsoft Entra ID using oauth2-proxy as a sidecar.

Final target state:

- Public users reach the app only through port 80.
- oauth2-proxy validates Entra bearer tokens.
- The FastAPI backend listens on port 8000 only for local/proxy use.
- Anonymous requests return 401 Unauthorized.
- Invalid bearer tokens return 403 Forbidden.
- Valid Entra bearer tokens return 200 OK.

Environment Summary

- Mac: used for Azure Portal, Azure CLI, SSH, and public curl tests.
- Ubuntu VM: <public-vm-ip>
- Tenant ID: <tenant-id>
- App registration / Client ID: <api-client-id>
- Application ID URI: api://<api-client-id>
- API scope: access_as_user

Architecture

Traffic flow:

1. Public client sends request to `http://<public-vm-ip>/entries`.
2. oauth2-proxy listens on port 80.
3. oauth2-proxy validates the bearer token against Microsoft Entra ID.
4. If valid, it forwards traffic to FastAPI at `http://127.0.0.1:8000`.
5. FastAPI returns application data.

Part 1: Azure App Registration

Step 1. Create the API app registration

Where: Azure Portal on the Mac

Why: Azure must know your API exists before it can issue tokens for it.

Configuration used:

- Name: LearningSteps-API
- Supported account types: My organization only
- Client ID: <api-client-id>
- Tenant ID: <tenant-id>

Step 2. Configure token audience and API scope

Where: Azure Portal on the Mac

Why: oauth2-proxy needs a stable audience and scope to validate JWTs for this API.

Actions taken:

1. Set Application ID URI to:

`api://<api-client-id>`

2. Added scope:

`access_as_user`

3. Added Azure CLI as an authorized client application:

`<azure-cli-public-client-id>`

Why the authorized client mattered:

- Without it, Azure CLI could not get tokens for the API and returned a consent error.

Step 3. Add redirect URI and client secret

Where: Azure Portal on the Mac

Why: oauth2-proxy requires a client ID and client secret. The app registration also requires a valid redirect URI format.

Redirect URI used:

`http://localhost/oauth2/callback`

Note:

- Azure rejected `http://<public-vm-ip>/oauth2/callback` because `http` with a raw public IP is not accepted for a web redirect URI.
- `http://localhost/oauth2/callback` was sufficient for the app registration because the main lab path used bearer tokens rather than browser sign-in.

Part 2: VM Access and Backend Verification

Step 4. Confirm the correct VM

Where: Azure Portal and Mac

Why: The proxy had to be installed on the same VM that already hosted the FastAPI app.

VM used:

- VM name: UbuntuOne
- Public IP: `<public-vm-ip>`

Step 5. Access the VM

Where: Azure Bastion and later SSH

Why: The VM had to be configured directly.

Important identity distinction:

- Azure sign-in account was not the Linux VM username.
- The VM account used was `<vm-admin-user>`.

Step 6. Verify the backend app on port 8000

Where: UbuntuOne VM

Why: `oauth2-proxy` cannot protect an app that is not running.

Useful commands:

```
curl -i http://127.0.0.1:8000/entries
```

```
sudo ss -ltnp | grep 8000
```

When the backend was not running, it was started with:

```
cd ~/learningsteps-api
nohup ./venv/bin/python -m uvicorn main:app --host 0.0.0.0 --port 8000
    > uvicorn.log 2>&1 &
```

Verification command:

```
curl -i http://127.0.0.1:8000/entries
```

Expected result:

- 200 OK

Part 3: Install oauth2-proxy

Step 7. Install the binary

Where: UbuntuOne VM

Why: oauth2-proxy is the security sidecar validating the JWTs.

Commands used:

```
LATEST=$(curl -fsSL https://api.github.com/repos/oauth2-proxy/oauth2-proxy/releases/latest | grep "tag_name" | cut -d '"' -f 4)
cd /tmp
curl -fLO "https://github.com/oauth2-proxy/oauth2-proxy/releases/download/${LATEST}/oauth2-proxy-${LATEST}.linux-amd64.tar.gz"
tar -xzf "oauth2-proxy-${LATEST}.linux-amd64.tar.gz"
sudo install -m 0755 "oauth2-proxy-${LATEST}.linux-amd64/oauth2-proxy" /usr/local/bin/oauth2-proxy
oauth2-proxy --version
```

Expected result:

- oauth2-proxy version output

Part 4: Configure oauth2-proxy

Step 8. Generate a valid cookie secret

Where: UbuntuOne VM

Why: oauth2-proxy requires a cookie secret with a valid length. The initial base64 string caused a startup failure.

Working command:

```
NEW_COOKIE_SECRET=$(openssl rand -hex 16)
echo "$NEW_COOKIE_SECRET"
```

Why this worked:

- It produced a 32-character secret accepted by oauth2-proxy.

Step 9. Create the config file

Where: UbuntuOne VM

Why: This file defines the OIDC provider, token validation behavior, upstream app, and listener port.

Final config file path:

/etc/oauth2-proxy/oauth2-proxy.cfg

Final working config:

```
provider = "oidc"
oidc_issuer_url = "https://login.microsoftonline.com/<tenant-id>/v2.0"

http_address = "0.0.0.0:80"
upstreams = [ "http://127.0.0.1:8000" ]

client_id = "<api-client-id>"
client_secret = "<Azure client secret value>"
redirect_url = "http://localhost/oauth2/callback"

cookie_secret = "<32-character hex secret>"
cookie_secure = false

email_domains = [ "*" ]

skip_jwt_bearer_tokens = true
extra_jwt_issuers = [ "https://sts.windows.net/<tenant-id>/api://<api-client-id>" ]
api_routes = [ ".*" ]
bearer_token_login_fallback = false
force_json_errors = true
```

Important note:

- The token issued by Azure CLI used issuer `https://sts.windows.net/.../`, not the expected `https://login.microsoftonline.com/.../v2.0`, so `extra_jwt_issuers` had to be updated to trust the actual issuer.

Part 5: Run oauth2-proxy as a Service

Step 10. Create the service account

Where: UbuntuOne VM

Why: Run oauth2-proxy as a dedicated system user.

Command:

```
sudo useradd --system --no-create-home --shell /usr/sbin/nologin
            oauth2-proxy || true
```

Step 11. Create the systemd service

Where: UbuntuOne VM

Why: Makes the proxy persistent and restartable.

File path:

/etc/systemd/system/oauth2-proxy.service

Final working service file:

[Unit]

Description=oauth2-proxy

After=network-online.target

Wants=network-online.target

[Service]

User=oauth2-proxy

Group=oauth2-proxy

AmbientCapabilities=CAP_NET_BIND_SERVICE

CapabilityBoundingSet=CAP_NET_BIND_SERVICE

ExecStart=/usr/local/bin/oauth2-proxy --config=/etc/oauth2-proxy/oauth2-proxy.cfg

Restart=always

RestartSec=5

[Install]

WantedBy=multi-user.target

Why the capabilities were needed:

- The proxy listens on port 80.
- Binding to port 80 requires privileged capability on Linux.

Start and verify commands:

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable --now oauth2-proxy
```

```
sudo systemctl status oauth2-proxy --no-pager
```

Expected result:

- active (running)

Part 6: Generate and Use Entra Tokens

Step 12. Set Mac-side variables

Where: Mac zsh

Why: reuse values in the token and test commands.

Commands:

```
export TENANT_ID=<tenant-id>
export APP_ID=<api-client-id>
export VM_IP=<public-vm-ip>
```

Step 13. Sign in with Azure CLI

Where: Mac zsh

Why: Azure CLI must authenticate in the correct tenant before requesting a token.

Command:

```
az login --tenant "$TENANT_ID"
```

Step 14. Request a token for the API

Where: Mac zsh

Why: obtain a bearer token scoped for the protected API.

Working command:

```
TOKEN=$(az account get-access-token \
  --scope "api://$APP_ID/access_as_user" \
  --query accessToken \
  -o tsv)
```

Check token length:

```
echo "${#TOKEN}"
```

Expected result:

- a large number, for example 1822

Step 15. Decode the token when debugging

Where: Mac zsh

Why: compare actual JWT issuer and audience with proxy expectations.

Commands:

```
export TOKEN
python3 - <<'PY'
import os, json, base64
token = os.environ["TOKEN"]
payload = token.split(".")[1]
payload += "=" * (-len(payload) % 4)
data = json.loads(base64.urlsafe_b64decode(payload))
print("iss:", data.get("iss"))
```

```
print("aud:", data.get("aud"))
print("scp:", data.get("scp"))
print("azp:", data.get("azp"))
PY
```

Observed values:

- iss: https://sts.windows.net/<tenant-id>/
- aud: api://<api-client-id>
- scp: access_as_user

Part 7: Functional Testing

Step 16. Anonymous test

Where: Mac zsh

Why: prove anonymous callers cannot use the API.

Command:

```
curl -i http://<public-vm-ip>/entries
```

Expected result:

- 401 Unauthorized

Step 17. Fake token test

Where: Mac zsh

Why: prove invalid JWTs are rejected.

Command:

```
curl -i -H "Authorization: Bearer fake-token" http://<public-vm-ip>/entries
```

Expected result:

- 403 Forbidden

Step 18. Real token test

Where: Mac zsh

Why: prove valid Entra bearer tokens are accepted.

Command:

```
curl -i -H "Authorization: Bearer $TOKEN" http://$VM_IP/entries
```


Expected result:

- 200 OK
- JSON body from FastAPI

Part 8: Lock Down Port 8000

Step 19. Identify the VM NSG

Where: Mac zsh

Why: remove public access to the backend port while leaving port 80 exposed.

Commands:

```
export VM_IP="<public-vm-ip>"
PUBLIC_IP_NAME=$(az network public-ip list --query "[?  
    ipAddress=='$VM_IP'] | [0].name" -o tsv)
RG=$(az network public-ip list --query "[?ipAddress=='$VM_IP'] |  
    [0].resourceGroup" -o tsv)
NIC_ID=$(az network public-ip show -g "$RG" -n "$PUBLIC_IP_NAME" --  
    query "ipConfiguration.id" -o tsv | sed  
    's#/ipConfigurations/.###')
NIC_NAME=${NIC_ID##*/}
NSG_ID=$(az network nic show -g "$RG" -n "$NIC_NAME" --query  
    "networkSecurityGroup.id" -o tsv)
NSG_NAME=${NSG_ID##*/}
NSG_RG=$(az resource show --ids "$NSG_ID" --query resourceGroup -o  
    tsv)
az network nsg rule list -g "$NSG_RG" --nsg-name "$NSG_NAME" -o table
```

Step 20. Delete the public rule for port 8000

Where: Mac zsh

Why: remove the back door.

Command:

```
az network nsg rule delete \  
    -g "$NSG_RG" \  
    --nsg-name "$NSG_NAME" \  
    -n "NAME-OF-8000-RULE"
```

Step 21. Verify only port 80 is public

Where: Mac zsh

Why: confirm final lockdown state.

Commands:

```
curl -i http://<public-vm-ip>/entries
```

```
curl -i http://<public-vm-ip>:8000/entries
```

Expected results:

- Port 80: 401 Unauthorized
- Port 8000: connection failure / timeout

Required Submission Screenshot

The required terminal evidence consisted of these three commands:

```
curl -i http://<public-vm-ip>/entries
```

```
curl -i -H "Authorization: Bearer fake-token" http://<public-vm-ip>/entries
```

```
curl -i -H "Authorization: Bearer $TOKEN" http://<public-vm-ip>/entries
```

Troubleshooting Log

Shell confusion: zsh vs PowerShell

Problem:

- export failed in PowerShell.

Reason:

- export is valid in zsh and Linux shells, not in PowerShell.

Fix:

- Use Mac zsh for Mac commands.
- Use Ubuntu shell for VM commands.

VM username confusion

Problem:

- Placeholder SSH commands used the wrong username.

Fix:

- Actual Linux username was <vm-admin-user>.

Cloud Shell vs VM shell

Problem:

- Azure Cloud Shell was mistaken for the VM shell.

Fix:

- Use Azure Bastion or native SSH for VM configuration.

Backend not listening on 8000

Problem:

- Local `curl` to `127.0.0.1:8000` failed.

Fix:

- Start `uvicorn` manually.
- Verify with `curl` and `ss`.

Bad GitHub release asset download

Problem:

- Hardcoded release URL returned a small invalid file.

Fix:

- Query the latest GitHub release tag dynamically and download that exact asset.

Invalid cookie secret length

Problem:

- `oauth2-proxy` refused the cookie secret.

Error:

`cookie_secret` must be 16, 24, or 32 bytes

Fix:

- Replace the old cookie secret with a valid 32-character hex string.

Broken systemd service file

Problem:

- Pasted heredoc content produced malformed service file lines.

Fix:

- Rewrite the service file cleanly and verify it with `cat`.

Port 80 bind permission denied

Problem:

- oauth2-proxy could not listen on port 80.

Error:

```
listen tcp 0.0.0.0:80: bind: permission denied
```

Fix:

- Add AmbientCapabilities=CAP_NET_BIND_SERVICE and CapabilityBoundingSet=CAP_NET_BIND_SERVICE to the service.

SSH / Bastion access broke

Problem:

- SSH and Bastion access failed because NSG rules blocked port 22.

Fix:

- Add a temporary inbound allow rule for TCP 22.
- Reconnect, then continue setup.

Azure CLI token consent error

Problem:

- Azure CLI could not request a token for the API.

Fix:

- Pre-authorize Azure CLI application ID 04b07795-8ddb-461a-bbee-02f9e1bf7b46 in Expose an API.

Real token returned 403

Problem:

- The JWT issuer did not match what oauth2-proxy expected.

Observed token values:

- iss = https://sts.windows.net/.../
- aud = api://<api-client-id>

Fix:

- Update extra_jwt_issuers to trust the sts.windows.net issuer with the correct audience.

Proxy accepted token but upstream failed

Problem:

- Authenticated request passed the proxy but failed to reach the backend.

Fix:

- Restart the backend app on port 8000.

Final Outcome

- External anonymous access: blocked with 401
- External invalid token access: blocked with 403
- External valid token access: allowed with 200
- Direct public access to port 8000: blocked
- App is protected by identity-aware gateway behavior on port 80

Cleanup Recommendations

1. Rotate the Azure client secret after submission because it was exposed during setup.
2. Keep both FastAPI and oauth2-proxy as proper services so the configuration survives reboot.
3. Remove any temporary SSH rule broader than necessary after the lab is finished.

Part 3 Completion Summary

Final Database Migration Outcome

The PostgreSQL migration was completed successfully with backup, recreate, and restore.

Final state:

- Original public PostgreSQL data was seeded and backed up to backup.sql.
- A new private PostgreSQL Flexible Server was created.
- The VM restored the seed data into the new private server.
- The FastAPI app was updated to use the new private PostgreSQL host.
- The app returned the restored seed rows after the migration.
- The database was no longer reachable from the Mac over the public internet.

New Private PostgreSQL Server

- Server name: <private-postgres-server-name>
- Resource group: <app-resource-group>
- Connectivity: Private access (VNet Integration)
- VNet: <virtual-network-name>
- Delegated subnet: <postgres-private-subnet>

- Private DB host: <private-postgres-host>

App Changes Required For Part 3

The original app was not DB-backed. It used a static Python list in `main.py`.

To complete Part 3 meaningfully, the app was first converted to PostgreSQL-backed behavior:

- Installed `sqlalchemy` and `psycopg`
- Replaced the in-memory entries list with PostgreSQL reads from an `entries` table
- Added DB connection environment variables to the `learningsteps-api systemd` service
- Restarted the service and verified the app returned DB data instead of static data

Commands Used For The Successful Restore

Where: UbuntuOne VM

Set the new target DB:

```
export NEW_PG_SERVER="<private-postgres-server-name>"
export PGPASSWORD='<database-password>'
```

Restore the backup:

```
psql "host=$NEW_PG_SERVER.postgres.database.azure.com port=5432
      dbname=postgres user=<db-admin-user> sslmode=require" <
      "$HOME/backup.sql"
```

Verify restored rows directly in the new private database:

```
psql "host=$NEW_PG_SERVER.postgres.database.azure.com port=5432
      dbname=postgres user=<db-admin-user> sslmode=require" -c
      "SELECT * FROM entries ORDER BY id;"
```

Update the app service to the new private host:

```
sudo sed -i.bak 's|^Environment=DB_HOST=.*|Environment=DB_HOST=
      <private-postgres-host>|' /etc/systemd/system/learningsteps-
      api.service
sudo systemctl daemon-reload
sudo systemctl restart learningsteps-api
sudo systemctl status learningsteps-api --no-pager
```

Verify the app locally on the VM:

```
curl -i http://127.0.0.1:8000/entries
```

Final Proof Commands

VM success proof:

```
psql "host=<private-postgres-host> port=5432 dbname=postgres user=<db-admin-user> sslmode=require" -c "SELECT * FROM entries ORDER BY id;"
```

Expected result:

- successful query output with the 5 seed rows

Mac failure proof:

```
$(brew --prefix libpq)/bin/psql "host=<private-postgres-host> port=5432 dbname=postgres user=<db-admin-user> sslmode=require"
```

Observed result:

- hostname resolution / connection failure from the Mac
- this proved the private database was not reachable publicly

Application integrity proof from the Mac through the Day 2 gatekeeper:

```
export TENANT_ID="<tenant-id>"
export APP_ID="<api-client-id>"
export VM_IP="<public-vm-ip>"
```

```
TOKEN=$(az account get-access-token \
  --scope "api://$APP_ID/access_as_user" \
  --query accessToken \
  -o tsv)
```

```
curl -i -H "Authorization: Bearer $TOKEN" http://$VM_IP/entries
```

Expected result:

- 200 OK
- all original seed rows returned through the protected application

Final Part 3 Success Criteria Met

- Backup created before deletion
- Backup content verified before migration
- New PostgreSQL server created with private access
- Database restored from the VM successfully
- App updated to the new private DB host
- VM query succeeded against the private database
- Mac query failed against the private database
- Protected app still returned the restored entries

Verified Final Results

Observed end state after the final repair steps:

- oauth2-proxy was restarted successfully after correcting the cookie secret length.
- learningsteps-api remained connected to the new private PostgreSQL host.

- Local VM application test returned the restored rows successfully.
- External application test from the Mac returned HTTP/1.1 200 OK with the restored seed rows.
- Direct PostgreSQL access from the Mac failed because the private database host was not publicly reachable/resolvable.

Observed protected app result from the Mac:

- HTTP/1.1 200 OK
- JSON body containing all 5 restored seed rows

Observed private database isolation result from the Mac:

- psql failed to connect or resolve the private PostgreSQL hostname

This confirmed the final target state:

- database isolated inside the private network
- application still functional through the identity-protected entrance

Class Part 5: Monitoring & Incident Response

Goal

Turn the protected edge into an observable, automated security control path:

- Nginx writes structured JSON logs.
- The VM ships those logs to Syslog.
- Azure Monitor Agent forwards Syslog into Log Analytics.
- Microsoft Sentinel detects repeated SQL injection probes.
- A Logic App playbook creates an NSG deny rule for the attacker IP.

Assumptions

- Nginx already fronts the application path described in the README.
- The VM is already onboarded to Azure and reachable through Azure Monitor Agent.
- Microsoft Sentinel will use the same subscription and region as the VM where possible.

Part 5.1: Structured Nginx Logging To Syslog

Step 1. Add a JSON log format

Where: UbuntuOne VM

Why: Sentinel detection works much better when the request fields are machine-parseable.

Update the main Nginx config:

File path:

/etc/nginx/nginx.conf

Add this log_format inside the http {} block:

```
log_format sentinel_json escape=json
'{
  '"time": "$time_iso8601", '
  '"remote_addr": "$remote_addr", '
  '"x_forwarded_for": "$http_x_forwarded_for", '
  '"request_id": "$request_id", '
  '"host": "$host", '
  '"method": "$request_method", '
  '"uri": "$request_uri", '
  '"protocol": "$server_protocol", '
  '"status": $status, '
  '"bytes_sent": $bytes_sent, '
  '"request_time": $request_time, '
  '"referer": "$http_referer", '
  '"user_agent": "$http_user_agent"'
}';
```

Why escape=json matters:

- It prevents quotes and special characters inside attacker payloads from breaking JSON parsing in Sentinel.

Step 2. Send Nginx access logs to Syslog

Where: UbuntuOne VM

Why: Azure Monitor Agent can collect Syslog directly without having to tail a custom file.

Update the active server block or site config:

Example file path:

/etc/nginx/sites-available/default

Add or replace the access log directive in the server {} block:

```
access_log
syslog:server=unix:/dev/log,facility=local7,tag=nginx,severity=info
sentinel_json;
error_log syslog:server=unix:/dev/log,facility=local7,tag=nginx_error
warn;
```

Validate and reload:

```
sudo nginx -t
sudo systemctl reload nginx
```

Step 3. Confirm logs arrive in local Syslog

Where: UbuntuOne VM

Why: This is the cheapest discriminating check before doing any Azure-side work.

Generate a test request:

```
curl -I http://127.0.0.1/
```

Check the journal or syslog stream:

```
sudo journalctl -t nginx -n 10 --no-pager
```

Expected result:

- one JSON log line containing keys like remote_addr, uri, and status

Part 5.2: Centralize Logs In Log Analytics And Microsoft Sentinel

Step 4. Create a Log Analytics Workspace

Where: Mac zsh or Azure Portal

Why: Sentinel stores analytics, incidents, and KQL data on top of a Log Analytics Workspace.

Example CLI:

```
export LOCATION="westeurope"
export MON_RG="learningsteps-monitoring-rg"
export LAW_NAME="learningsteps-law"

az group create -n "$MON_RG" -l "$LOCATION"

az monitor log-analytics workspace create \
  -g "$MON_RG" \
  -n "$LAW_NAME" \
  -l "$LOCATION"
```

Step 5. Onboard Microsoft Sentinel

Where: Azure Portal

Why: Sentinel provides analytics rules, incidents, automations, and workbooks.

Portal path:

1. Open Microsoft Sentinel.
2. Select Create.
3. Choose the workspace learningsteps-law.
4. Complete onboarding.

Step 6. Install Azure Monitor Agent on the VM

Where: Azure Portal or Mac zsh

Why: AMA is the supported path for shipping Syslog into the workspace via DCR.

If the VM does not already have AMA, install the extension:

```
export VM_RG="<app-resource-group>"
export VM_NAME="Ubuntu0ne"

az vm extension set \
  -g "$VM_RG" \
  --vm-name "$VM_NAME" \
  --publisher Microsoft.Azure.Monitor \
  --name AzureMonitorLinuxAgent
```

Step 7. Create and associate a Data Collection Rule

Where: Azure Portal

Why: The DCR tells AMA which Syslog facilities and severities to forward.

Recommended DCR settings:

- Data source type: Syslog
- Facility names: local7
- Log levels: Informational, Notice, Warning, Error, Critical, Alert, Emergency
- Destination: learningsteps-law
- Target resource: Ubuntu0ne

Portal path:

1. Open Monitor.
2. Select Data Collection Rules.
3. Create a new rule in the same region as the VM.
4. Add the Syslog data source above.
5. Add the Log Analytics Workspace destination.
6. Associate the DCR with the VM Ubuntu0ne.

Step 8. Validate cloud-side ingestion with KQL

Where: Log Analytics query window

Why: Confirm the telemetry path before building a detection rule.

Use this query:

```
Syslog
| where Facility =~ "local7"
| where ProcessName has "nginx"
| extend payload = parse_json(SyslogMessage)
```

```
| project TimeGenerated, Computer, ProcessName, Facility, payload
| sort by TimeGenerated desc
```

Expected result:

- recent Nginx access events with parsed JSON in payload

Part 5.3: Build The SQLi Hunter Analytics Rule

Step 9. Use this KQL detection query

Where: Microsoft Sentinel Analytics rule wizard

Why: The rule hunts for repeated blocked SQL injection attempts from the same IP.

KQL query:

```
let Lookback = 5m;
let Threshold = 10;
let SqlTerms = dynamic(["union", "select", "drop"]);
Syslog
| where TimeGenerated >= ago(Lookback)
| where Facility =~ "local7"
| where ProcessName has "nginx"
| extend payload = parse_json(SyslogMessage)
| extend ClientIp = tostring(payload.remote_addr)
| extend Request = tostring(payload.uri)
| extend Status = toint(payload.status)
| where Status == 403
| where Request has_any (SqlTerms)
| summarize AttemptCount = count(),
              FirstSeen = min(TimeGenerated),
              LastSeen = max(TimeGenerated),
              SampleRequests = make_set(Request, 5)
  by ClientIp
| where AttemptCount > Threshold
```

Why this shape works:

- `403` limits the rule to requests blocked at the edge.
- `has\_any` catches the obvious SQL keywords required by the lab.
- `summarize by ClientIp` turns noisy logs into one actionable incident per source.

If you want stricter matching, replace the keyword filter with regex:

```
`kusto
| where Request matches regex @"(?i)\b(union|select|drop)\b"
```

Step 10. Configure the scheduled analytics rule

Where: Microsoft Sentinel

Recommended settings:

- Rule name: SQLi Hunter – Repeated 403 From Single IP
- Query scheduling: every 5 minutes
- Lookup data from the last: 5 minutes
- Alert threshold: generate alert when query returns more than 0 results
- Incident creation: Enabled
- Event grouping: Group all events into a single alert per query run

Entity mapping:

- Map ClientIp to the IP entity

Alert details:

- Alert name format: SQLi Hunter blocked activity from {{ClientIp}}
- Severity: High
- Tactics: Initial Access, Reconnaissance

Part 5.4: Automated Response With A Logic App Playbook

Step 11. Prepare the NSG details

Where: Mac zsh

Why: The playbook needs the exact NSG resource ID.

Use the same public IP already documented in this runbook:

```
export VM_IP="<public-vm-ip>"
PUBLIC_IP_NAME=$(az network public-ip list --query "[?
    ipAddress=='$VM_IP'] | [0].name" -o tsv)
RG=$(az network public-ip list --query "[?ipAddress=='$VM_IP'] |
    [0].resourceGroup" -o tsv)
NIC_ID=$(az network public-ip show -g "$RG" -n "$PUBLIC_IP_NAME" --
    query "ipConfiguration.id" -o tsv | sed
    's#/ipConfigurations/.*/##')
NIC_NAME=${NIC_ID##*/}
NSG_ID=$(az network nic show -g "$RG" -n "$NIC_NAME" --query
    "networkSecurityGroup.id" -o tsv)
NSG_NAME=${NSG_ID##*/}
NSG_RG=$(az resource show --ids "$NSG_ID" --query resourceGroup -o
    tsv)

printf 'NSG_RG=%s\nNSG_NAME=%s\nNSG_ID=%s\n' "$NSG_RG" "$NSG_NAME"
"$NSG_ID"
```

Step 12. Create the playbook

Where: Azure Portal

Why: Sentinel uses playbooks for incident-triggered automation.

Create a Logic App with these characteristics:

- Type: Consumption
- Name: sentinel-block-attacker-ip
- Resource group: learningsteps-monitoring-rg
- Enable system-assigned managed identity: Yes

Required RBAC:

- Grant the Logic App managed identity Network Contributor on the VM NSG.
- Grant the account linking the playbook in Sentinel enough rights to run playbooks from Sentinel.

Step 13. Build the workflow

Where: Logic App Designer

Use this control flow:

1. Trigger: Microsoft Sentinel incident
2. Action: Get incident using the incident ARM ID from the trigger
3. Action: Entities – Get IPs from incident or equivalent entity expansion step
4. Action: For each IP entity
5. Action: HTTP with Managed Identity to create or update an NSG deny rule

Use the managed identity in the HTTP action:

- Authentication type: Managed identity
- Audience: <https://management.azure.com/>
- Method: PUT

URI template:

```
https://management.azure.com/subscriptions/<subscription-id>/resourceGroups/<nsg-resource-group>/providers/Microsoft.Network/networkSecurityGroups/<nsg-name>/securityRules/Deny-SQLi-@{replace(items('For_each_IP'),'Address'),' ','-')}?api-version=2023-09-01
```

Example request body:

```
{
  "properties": {
    "priority": 300,
    "access": "Deny",
    "direction": "Inbound",
    "protocol": "*",
    "sourcePortRange": "*",
```

```

    "destinationPortRange": "*",
    "sourceAddressPrefix": "@{items('For_each_IP')?['Address']}",
    "destinationAddressPrefix": "*",
    "description": "Auto-block created by Microsoft Sentinel SQLi Hunter"
  }
}

```

Important implementation note:

- NSG rule priorities must be unique. If 300 is already used, choose a lower-priority-number gap such as 301 or 310 and keep it reserved for Sentinel automation.

Step 14. Link the playbook to the analytics rule

Where: Microsoft Sentinel

Why: The incident must trigger the kill-switch automatically.

Create an automation rule:

1. Open Microsoft Sentinel.
2. Select Automation.
3. Create rule Auto-block SQLi source IP.
4. Condition: incident title contains SQLi Hunter.
5. Action: run playbook sentinel-block-attacker-ip.

Part 5.5: Testing The Attack And Block

Step 15. Launch a repeat SQLi probe from the Mac

Where: Mac zsh

Why: The analytics rule threshold requires more than 10 hits in 5 minutes.

Example test loop:

```

export VM_IP="<public-vm-ip>"

for i in {1..15}; do
  curl -s -o /dev/null -w "%{http_code}\n" \
    "http://$VM_IP/?id=1%20UNION%20SELECT%20password%20FROM%20users"
done

```

Expected result:

- repeated 403 responses from Nginx or the WAF layer

Step 16. Watch the incident appear in Sentinel

Where: Microsoft Sentinel portal

Open:

- Incidents
- Analytics
- Automation

Expected timeline:

- logs arrive in Log Analytics within a few minutes
- the scheduled rule runs on the next cycle
- a new incident appears titled similar to SQLi Hunter blocked activity from <your-ip>
- the automation rule runs the playbook

Step 17. Verify the NSG deny rule was created

Where: Azure Portal or Mac zsh

Portal proof:

1. Open the VM NSG.
2. Select Inbound security rules.
3. Confirm a new rule named like Deny-SQLi-<your-ip> exists.

CLI proof:

```
az network nsg rule list \
  -g "$NSG_RG" \
  --nsg-name "$NSG_NAME" \
  --query "[?starts_with(name, 'Deny-SQLi-')]" \
  -o table
```

Step 18. Verify connectivity is blocked

Where: Mac zsh

Reliable proof command:

```
curl -i --connect-timeout 5 "http://$VM_IP/"
```

Expected result:

- timeout or failed TCP connection after the deny rule is active

Important note:

- ping is not always a reliable proof because ICMP may already be disabled or filtered independently of the HTTP path. Use failed HTTP connectivity as the main submission proof.

Part 5 Submission Evidence

Required Screenshot 1: The Incident

Capture the Microsoft Sentinel Incidents page showing:

- incident title with the SQLi Hunter name
- severity
- status
- incident creation time

Required Screenshot 2: The Response

Capture the Azure NSG Inbound security rules page showing:

- the auto-created Deny-SQLi-<your-ip> rule
- the blocked source IP
- the deny action
- the rule priority

Bonus Challenge Notes

Geographic visualization workbook

If public IP enrichment is enabled, start with this workbook query:

```
Syslog
| where Facility =~ "local7"
| where ProcessName has "nginx"
| extend payload = parse_json(SyslogMessage)
| extend ClientIp = tostring(payload.remote_addr)
| extend Status = toint(payload.status)
| where Status == 403
| summarize Count = count() by ClientIp
^^^
```

Then add a map visualization and enrich IPs through Sentinel watchlists or threat intelligence connectors if needed.

Approval before blocking

Modify the playbook flow:

1. Insert an approval step before the NSG PUT action.
2. Send the approval to Microsoft Teams or Slack.
3. Only create the deny rule when the response is Approve.

Part 5 Troubleshooting

Logs never show up in Log Analytics

Check:

- Nginx is actually writing to `local7`
- the DCR is associated with the VM
- the Azure Monitor Agent extension is installed and healthy
- the query window is looking at the Syslog table in the correct workspace

KQL returns rows in Log Analytics but no incident is created

Check:

- the scheduled rule interval matches the lab timing
- entity mapping uses `ClientIp`
- incident creation is enabled in the analytics rule
- the query really returns `AttemptCount > 10`

Playbook runs but no NSG rule is created

Check:

- the Logic App managed identity has `Network Contributor` on the NSG
- the ARM PUT URL uses the correct subscription, resource group, and NSG name
- the chosen NSG priority is not already in use
- the incident actually contains an IP entity

The deny rule appears but your requests still work

Check:

- your Mac public IP did not change during testing
- the NSG is attached to the active NIC or subnet in the traffic path
- another higher-priority allow rule is not overriding the new deny rule

Part 5 Final Outcome

When successful, the end state should be:

- Nginx writes structured JSON logs into Syslog.
- Azure Monitor Agent forwards those logs into Log Analytics.
- Microsoft Sentinel raises an incident when one IP triggers more than 10 blocked SQLi-style requests in 5 minutes.
- A Logic App automatically adds an NSG deny rule for the offending IP.

- The attacking client loses HTTP connectivity to the VM within minutes.